

TCS NQT DIGITAL CODING ROUND

Complete Self-Study Manual

Covers: Arrays · Numbers · Number Systems · Sorting · Strings
Languages: Java (primary) · C++ · Python

All 100+ problems from the official TCS NQT Coding Sheet — fully solved

Edition 2024-25

How to Use This Manual

This manual is designed as a complete self-contained textbook for the TCS NQT Digital Coding Round. It does not assume prior competitive programming experience, only a working knowledge of any one programming language (Java, C++, or Python). Every concept is explained from the ground up, followed by carefully worked examples and then real exam-style practice problems.

Structure of Each Chapter

- Concept Explanation — Plain-English description of the idea and why it matters.
- Worked Examples — Step-by-step walkthroughs with diagrams in text form.
- Java Code — Production-quality, fully commented Java implementation.
- C++ / Python Notes — Key differences and equivalent built-ins.
- Edge Cases & Tricks — What trips up candidates in the actual exam.
- Practice Problem Set — 5–10 problems with hints; try before reading hints.

Key Algorithmic Patterns Covered

Throughout this manual, you will encounter these recurring patterns. Recognising which pattern applies is often the hardest part of the exam, so each chapter highlights the pattern name explicitly.

- Two-Pointer Technique — for array/string problems requiring pair detection.
- Sliding Window — for sub-array/substring problems with a fixed or variable window.
- Hashing (HashMap/HashSet) — for frequency counting, duplicate detection, lookups in $O(1)$.
- Prefix Sums — for range-sum queries and equilibrium problems.
- Sorting as a Pre-step — many problems become trivial after sorting.
- Recursion & Divide-and-Conquer — for problems that decompose naturally (factorial, Merge Sort, Quick Sort).
- String Manipulation — character arrays, StringBuilder, ASCII arithmetic.

Language Tip

All code in this manual is written in Java, the most common language chosen by TCS NQT candidates. At the end of major code blocks you will find brief C++ and Python equivalents for the key lines. Java's Scanner, Arrays, Collections, StringBuilder, and HashMap classes are used extensively — these are explained in detail inside each chapter where they first appear.

EXAM QUICK FACTS

TCS NQT Coding Round: 2 problems, 60 minutes.

Difficulty: Easy to Medium (think LeetCode Easy/Medium).

Languages allowed: Java, C, C++, Python, Perl.

Partial marks are awarded — even a 50% solution scores points.

Focus areas: Arrays, Strings, Math/Numbers, Sorting, Basic Data Structures.

Chapter 1: Arrays

An array is the most fundamental data structure in programming. It stores a fixed-size collection of elements of the same type in contiguous memory. Every TCS NQT exam has at least one array problem, so mastering this chapter is non-negotiable.

1.0 Array Basics in Java

In Java, you declare and initialise an array like this:

```
// Declaration
int[] arr;

// Initialisation with size
arr = new int[5];           // {0, 0, 0, 0, 0} by default

// Declaration + initialisation at once
int[] arr = {10, 30, 20, 50, 40};

// Length
int n = arr.length;        // 5

// Accessing elements (0-indexed)
System.out.println(arr[0]); // 10
System.out.println(arr[4]); // 40

// Iterating
for (int i = 0; i < arr.length; i++) {
    System.out.print(arr[i] + " ");
}

// Or using enhanced for-loop
for (int x : arr) {
    System.out.print(x + " ");
}
```

IMPORTANT — Arrays.sort()

Java's Arrays.sort() uses Dual-Pivot Quicksort for primitives (avg $O(n \log n)$).

Import: import java.util.Arrays;

Usage: Arrays.sort(arr); // sorts entire array ascending

Arrays.sort(arr, 2, 5); // sorts indices 2,3,4 only

Arrays.sort(arr); // for reverse, sort then reverse manually

1.1 Smallest and Largest Element

Strategy: Initialise min/max to the first element, then scan the rest of the array comparing each element.

Why the first element and not `Integer.MAX_VALUE`? Both work, but using `arr[0]` avoids the edge case where the array is all-negative (`Integer.MAX_VALUE` would never get replaced if every value is negative — actually it would, but using `arr[0]` is cleaner and self-documenting).

```
import java.util.Arrays;

public class MinMax {
    public static void main(String[] args) {
        int[] arr = {5, 2, 8, 1, 9, 3};

        // --- Method 1: Manual loop ---
        int min = arr[0], max = arr[0];
        for (int i = 1; i < arr.length; i++) {
            if (arr[i] < min) min = arr[i];
            if (arr[i] > max) max = arr[i];
        }
        System.out.println("Min = " + min); // 1
        System.out.println("Max = " + max); // 9

        // --- Method 2: Sort (modifies original array) ---
        Arrays.sort(arr);
        System.out.println("Min = " + arr[0]);
        System.out.println("Max = " + arr[arr.length - 1]);
    }
}
```

Time Complexity: $O(n)$ for manual, $O(n \log n)$ for sort. Space: $O(1)$ for both.

EDGE CASES

Single element array: `arr = {7}` → `min = max = 7`. Works fine with the loop.

All equal elements: `arr = {5,5,5}` → `min = max = 5`.

Negative numbers: `arr = {-5,-1,-3}` → `min = -5, max = -1`. Works fine.

1.2 Second Smallest and Second Largest

You need to track two variables for min and two for max. A common mistake is confusing 'second minimum' with 'minimum of elements excluding the overall minimum'. They are the same thing, but you need a single-pass solution that handles duplicates gracefully.

Algorithm (single pass):

Step 1: Set `min1 = arr[0]`, `min2 = Integer.MAX_VALUE` (similarly `max1, max2 = Integer.MIN_VALUE`).

Step 2: For each element `x`: if `x < min1`, then `min2 = min1` and `min1 = x`. Else if `x < min2` and `x != min1`, then `min2 = x`.

Step 3: Apply mirror logic for max.

```
public class SecondMinMax {
```

```

public static void main(String[] args) {
    int[] arr = {12, 13, 1, 10, 34, 1};
    int min1 = Integer.MAX_VALUE, min2 = Integer.MAX_VALUE;
    int max1 = Integer.MIN_VALUE, max2 = Integer.MIN_VALUE;

    for (int x : arr) {
        // Second minimum
        if (x < min1) { min2 = min1; min1 = x; }
        else if (x < min2 && x != min1) { min2 = x; }

        // Second maximum
        if (x > max1) { max2 = max1; max1 = x; }
        else if (x > max2 && x != max1) { max2 = x; }
    }

    System.out.println("Second Smallest = " + min2); // 10
    System.out.println("Second Largest = " + max2); // 13
}

```

TRICK

Some questions ask for second smallest DISTINCT value (ignore duplicates).

Others ask for the element at the second position if the array were sorted (duplicates allowed).

Read the problem carefully! The `x != min1` guard in the code above handles the distinct-value case.

1.3 Reverse an Array

Use the two-pointer technique: one pointer starts at the beginning, the other at the end. Swap elements and move pointers toward each other.

```

public class ReverseArray {
    public static void reverse(int[] arr) {
        int left = 0, right = arr.length - 1;
        while (left < right) {
            // Swap without a temp variable (XOR trick)
            arr[left] ^= arr[right];
            arr[right] ^= arr[left];
            arr[left] ^= arr[right];
            left++;
            right--;
        }
    }

    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 4, 5};
        reverse(arr);
    }
}

```

```
// arr is now {5, 4, 3, 2, 1}
for (int x : arr) System.out.print(x + " ");
}
}
```

The XOR swap is space-optimal and avoids a temporary variable. However, if `left == right` (odd-length array, middle element), XOR-ing a number with itself gives 0, which would destroy the element — but the loop condition `left < right` prevents this case.

NOTE — String Reversal

For strings, Java's `StringBuilder.reverse()` does this in one line:

```
String rev = new StringBuilder(str).reverse().toString();
```

For arrays: `Arrays` has no built-in reverse; use `Collections.reverse(list)` on a `List`.

1.4 Count Frequency of Each Element

Pattern: Hashing. Use a `HashMap<Integer, Integer>` to store element → count.

```
import java.util.*;

public class Frequency {
    public static void main(String[] args) {
        int[] arr = {2, 3, 2, 5, 3, 3, 1};
        Map<Integer, Integer> freq = new HashMap<>();

        for (int x : arr) {
            freq.put(x, freq.getOrDefault(x, 0) + 1);
        }

        // Print frequencies
        for (Map.Entry<Integer, Integer> e : freq.entrySet()) {
            System.out.println(e.getKey() + " -> " + e.getValue());
        }

        // Output (order may vary): 1->1, 2->2, 3->3, 5->1
    }
}
```

`getOrDefault(key, 0)` is the idiomatic Java way to avoid null-pointer exceptions when the key has not been seen before. It returns the mapped value if the key exists, otherwise returns the default (0).

1.5 Rearrange Array in Increasing-Decreasing Order

Sort ascending first. Then, if descending order is needed, either reverse the sorted array or sort a boxed `Integer[]` with a reverse comparator.

```
import java.util.*;

public class SortOrder {
    public static void main(String[] args) {
```

```

int[] arr = {5, 2, 8, 1, 9, 3};

// Ascending
Arrays.sort(arr);
System.out.println("Ascending: " + Arrays.toString(arr));

// Descending – reverse the sorted array
for (int l=0, r=arr.length-1; l<r; l++, r--) {
    int tmp = arr[l]; arr[l] = arr[r]; arr[r] = tmp;
}
System.out.println("Descending: " + Arrays.toString(arr));

// Descending using Integer[] and Comparator
Integer[] arr2 = {5, 2, 8, 1, 9, 3};
Arrays.sort(arr2, Collections.reverseOrder());
System.out.println("Descending: " + Arrays.toString(arr2));
}
}

```

1.6 Sum and Average

Simple linear scan. Beware of integer overflow when summing large arrays: use a long instead of int for the accumulator.

```

public class SumAverage {
    public static void main(String[] args) {
        int[] arr = {10, 20, 30, 40, 50};

        long sum = 0;           // long to avoid overflow
        for (int x : arr) sum += x;

        double avg = (double) sum / arr.length;
        System.out.println("Sum = " + sum);           // 150
        System.out.println("Avg = " + avg);           // 30.0
    }
}

```

OVERFLOW RULE

int max value ≈ 2.1 billion ($2^{31} - 1$).

If the array has up to 10^5 elements each up to 10^9 , sum can reach 10^{14} — use long.

Cast to double only AFTER computing the sum: $(\text{double}) \text{sum} / n$.

1.7 Median of an Array

The median is the middle value when the array is sorted.

- If n is ODD: median = sorted[$n/2$]

- If n is EVEN: median = (sorted[n/2 - 1] + sorted[n/2]) / 2.0

```
import java.util.Arrays;

public class Median {
    public static double findMedian(int[] arr) {
        Arrays.sort(arr);
        int n = arr.length;
        if (n % 2 == 1) {
            return arr[n / 2];
        } else {
            return (arr[n/2 - 1] + arr[n/2]) / 2.0;
        }
    }

    public static void main(String[] args) {
        System.out.println(findMedian(new int[]{3,1,2})); // 2.0
        System.out.println(findMedian(new int[]{4,1,2,3})); // 2.5
    }
}
```

1.8 Remove Duplicates

1.8a From a Sorted Array (In-Place, O(1) Space)

Because the array is sorted, duplicates are adjacent. Use two pointers: i tracks the last unique element's position; j scans ahead.

```
public class RemoveDupSorted {
    // Returns length of new array (first 'length' elements are unique)
    public static int removeDups(int[] arr) {
        if (arr.length == 0) return 0;
        int i = 0; // slow pointer
        for (int j = 1; j < arr.length; j++) { // fast pointer
            if (arr[j] != arr[i]) {
                i++;
                arr[i] = arr[j];
            }
        }
        return i + 1; // new length
    }

    public static void main(String[] args) {
        int[] arr = {1, 1, 2, 2, 3, 4, 4};
        int len = removeDups(arr);
        // First 'len' elements: {1, 2, 3, 4}
        System.out.println("Unique count: " + len);
    }
}
```

}

1.8b From an Unsorted Array (Using HashSet)

```
import java.util.*;

public class RemoveDupUnsorted {
    public static void main(String[] args) {
        int[] arr = {3, 1, 4, 1, 5, 9, 2, 6, 5, 3};
        Set<Integer> seen = new LinkedHashSet<>(); // preserves order
        for (int x : arr) seen.add(x);

        System.out.println("Without duplicates: " + seen);
        // [3, 1, 4, 5, 9, 2, 6]
    }
}
```

LinkedHashSet maintains insertion order. HashSet does not guarantee order. TreeSet sorts the elements. Choose based on the required output.

1.9 Repeating and Non-Repeating Elements

Use a HashMap to count frequencies, then filter.

```
import java.util.*;

public class RepeatingNonRepeating {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3, 1, 2, 4};
        Map<Integer, Integer> freq = new HashMap<>();
        for (int x : arr) freq.put(x, freq.getOrDefault(x, 0) + 1);

        System.out.print("Repeating: ");
        for (Map.Entry<Integer, Integer> e : freq.entrySet())
            if (e.getValue() > 1) System.out.print(e.getKey() + " ");

        System.out.print("\nNon-Repeating: ");
        for (Map.Entry<Integer, Integer> e : freq.entrySet())
            if (e.getValue() == 1) System.out.print(e.getKey() + " ");
    }
}
```

1.10 Symmetric Pairs in an Array

A pair (a, b) and (b, a) are symmetric. Given an array of pairs, find all symmetric pairs.

Strategy: Use a HashMap<Integer, Integer> where key = first element of pair, value = second element. For each pair (x, y), check if the map contains y as a key with value x.

```
import java.util.*;

public class SymmetricPairs {
    public static void main(String[] args) {
        int[][] pairs = {{1,2},{3,4},{5,6},{2,1},{4,3}};
        Map<Integer,Integer> map = new HashMap<>();

        for (int[] pair : pairs) {
            int x = pair[0], y = pair[1];
            if (map.containsKey(y) && map.get(y) == x) {
                System.out.println("(" + x + "," + y + ") and (" + y + "," + x + ")");
            } else {
                map.put(x, y);
            }
        }
    }
}
```

1.11 Maximum Product Subarray

This problem is famous because negative numbers can flip the sign: two negatives multiply to a positive, which could be the maximum. Therefore, we must track BOTH the current maximum and current minimum product.

Algorithm (Kadane's variant for products):

Step 1: Initialise $\text{maxProd} = \text{minProd} = \text{result} = \text{arr}[0]$.

Step 2: For each subsequent element x , compute $\text{tempMax} = \max(x, \text{maxProd} * x, \text{minProd} * x)$.

Step 3: Update $\text{minProd} = \min(x, \text{maxProd} * x, \text{minProd} * x)$. Then $\text{maxProd} = \text{tempMax}$.

Step 4: Update $\text{result} = \max(\text{result}, \text{maxProd})$.

```
public class MaxProductSubarray {
    public static int maxProduct(int[] arr) {
        int maxProd = arr[0], minProd = arr[0], result = arr[0];

        for (int i = 1; i < arr.length; i++) {
            int x = arr[i];
            int tempMax = Math.max(x, Math.max(maxProd * x, minProd * x));
            minProd = Math.min(x, Math.min(maxProd * x, minProd * x));
            maxProd = tempMax;
            result = Math.max(result, maxProd);
        }
        return result;
    }

    public static void main(String[] args) {
```

```
System.out.println(maxProduct(new int[]{2,3,-2,4})); // 6
System.out.println(maxProduct(new int[]{-2,0,-1})); // 0
System.out.println(maxProduct(new int[]{-2,-3,-4})); // 12 (-3*-4)
}
}
```

1.12 Replace Each Element by Its Rank

The rank of an element is its position in the sorted order (1-indexed). Equal elements get the same rank.

```
import java.util.*;

public class RankReplace {
    public static void replaceByRank(int[] arr) {
        int n = arr.length;
        int[] sorted = arr.clone();
        Arrays.sort(sorted);

        // Map element -> rank
        Map<Integer,Integer> rankMap = new HashMap<>();
        int rank = 1;
        for (int x : sorted) {
            if (!rankMap.containsKey(x)) rankMap.put(x, rank++);
        }

        // Replace
        for (int i = 0; i < n; i++) arr[i] = rankMap.get(arr[i]);
    }

    public static void main(String[] args) {
        int[] arr = {20, 15, 26, 2, 98, 6};
        replaceByRank(arr);
        System.out.println(Arrays.toString(arr));
        // [4, 3, 5, 1, 6, 2]
    }
}
```

1.13 Sort Elements by Frequency

Sort such that the most frequent element comes first. For equal frequencies, sort by first occurrence or value as specified.

```
import java.util.*;

public class SortByFrequency {
    public static void main(String[] args) {
```

```

int[] arr = {2, 3, 2, 4, 5, 12, 2, 3, 3, 3, 12};
Map<Integer,Integer> freq = new LinkedHashMap<>();
for (int x : arr) freq.put(x, freq.getOrDefault(x,0)+1);

// Get keys list and sort by frequency descending
List<Integer> keys = new ArrayList<>(freq.keySet());
keys.sort((a, b) -> freq.get(b) - freq.get(a));

for (int k : keys)
    for (int i = 0; i < freq.get(k); i++)
        System.out.print(k + " ");
// 3 3 3 3 2 2 2 12 12 4 5
}
}

```

1.14 Rotation of Array — Left and Right

Rotating an array left by k positions means the first k elements move to the end. A clever trick uses three reversals:

- Left rotate by k: reverse[0..k-1], reverse[k..n-1], reverse[0..n-1]
- Right rotate by k: left rotate by (n - k)

```

import java.util.Arrays;

public class RotateArray {
    private static void reverse(int[] a, int l, int r) {
        while (l < r) { int t = a[l]; a[l]=a[r]; a[r]=t; l++; r--; }
    }

    public static void leftRotate(int[] a, int k) {
        int n = a.length;
        k %= n; // handle k >= n
        reverse(a, 0, k-1);
        reverse(a, k, n-1);
        reverse(a, 0, n-1);
    }

    public static void rightRotate(int[] a, int k) {
        leftRotate(a, a.length - k % a.length);
    }

    public static void main(String[] args) {
        int[] arr = {1,2,3,4,5};
        leftRotate(arr.clone(), 2); // {3,4,5,1,2}
        rightRotate(arr.clone(), 2); // {4,5,1,2,3}
        int[] a = {1,2,3,4,5};
    }
}

```

```

        leftRotate(a, 2);
        System.out.println(Arrays.toString(a)); // [3, 4, 5, 1, 2]
    }
}

```

BLOCK SWAP ALGORITHM

The Block Swap Algorithm is another $O(n)$ rotation method:

- Split array into A (first k) and B (rest).
- If $|A| == |B|$: swap them and done.
- If $|A| < |B|$: swap A with first $|A|$ elements of B, then recurse on remaining B.
- If $|A| > |B|$: swap last $|B|$ elements of A with B, then recurse on remaining A.

For TCS NQT, the 3-reversal method is simpler to implement correctly under time pressure.

1.15 Equilibrium Index (Prefix Sum)

An index i is an equilibrium index if the sum of elements to the left equals the sum to the right. Pattern: Prefix Sums.

Step 1: Compute totalSum of all elements.

Step 2: Traverse the array, maintaining leftSum. At index i : $\text{rightSum} = \text{totalSum} - \text{leftSum} - \text{arr}[i]$. If $\text{leftSum} == \text{rightSum}$, i is equilibrium.

```

public class EquilibriumIndex {
    public static int findEquilibrium(int[] arr) {
        long total = 0;
        for (int x : arr) total += x;

        long left = 0;
        for (int i = 0; i < arr.length; i++) {
            long right = total - left - arr[i];
            if (left == right) return i;
            left += arr[i];
        }
        return -1; // no equilibrium
    }

    public static void main(String[] args) {
        System.out.println(findEquilibrium(new int[]{-7,1,5,2,-4,3,0})); // 3
    }
}

```

1.16 Circular Rotation by K Positions

Circular rotation is the same as array rotation. The 3-reversal trick handles this perfectly. Just ensure $k = k \% n$ to handle $k \geq n$.

1.17 Sort an Array According to Another Array's Order

Given two arrays A and B, sort A such that relative order of elements defined in B comes first, remaining elements sorted normally.

```
import java.util.*;

public class SortByOrder {
    public static void main(String[] args) {
        int[] A = {2,1,2,5,7,1,9,3,6,8,8};
        int[] B = {2,1,8,3};

        // Assign order positions using map
        Map<Integer,Integer> order = new HashMap<>();
        for (int i = 0; i < B.length; i++) order.put(B[i], i);

        // Count frequencies in A
        Map<Integer,Integer> freq = new HashMap<>();
        for (int x : A) freq.put(x, freq.getOrDefault(x,0)+1);

        List<Integer> result = new ArrayList<>();

        // First, add elements in B's order
        for (int b : B) {
            if (freq.containsKey(b)) {
                for (int i = 0; i < freq.get(b); i++) result.add(b);
                freq.remove(b);
            }
        }

        // Remaining elements sorted
        List<Integer> rest = new ArrayList<>(freq.keySet());
        Collections.sort(rest);
        for (int k : rest)
            for (int i = 0; i < freq.get(k); i++) result.add(k);

        System.out.println(result);
        // [2, 2, 1, 1, 8, 8, 3, 5, 6, 7, 9]
    }
}
```

1.18 Search an Element in an Array

Linear Search: $O(n)$. Binary Search: $O(\log n)$ — requires sorted array.

```
import java.util.Arrays;

public class Search {
```

```
// Linear Search
public static int linearSearch(int[] arr, int target) {
    for (int i = 0; i < arr.length; i++)
        if (arr[i] == target) return i;
    return -1;
}

// Binary Search (array must be sorted)
public static int binarySearch(int[] arr, int target) {
    int l = 0, r = arr.length - 1;
    while (l <= r) {
        int mid = l + (r - l) / 2; // prevents overflow
        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) l = mid + 1;
        else r = mid - 1;
    }
    return -1;
}

public static void main(String[] args) {
    int[] arr = {1, 3, 5, 7, 9, 11};
    System.out.println(linearSearch(arr, 7)); // 3
    System.out.println(binarySearch(arr, 7)); // 3
    System.out.println(Arrays.binarySearch(arr, 7)); // built-in: 3
}
}
```

OVERFLOW IN BINARY SEARCH

Never write $\text{mid} = (l + r) / 2$ — if l and r are large, $l+r$ overflows int.

Always use $\text{mid} = l + (r - l) / 2$.

`Arrays.binarySearch()` is the Java built-in. It returns a **NEGATIVE** value if not found.

1.19 Check if Array is Subset of Another Array

Put all elements of the larger array into a `HashSet`. Then check that every element of the smaller array is in the set.

```
import java.util.*;

public class SubsetCheck {
    public static boolean isSubset(int[] A, int[] B) {
        // Check if B is a subset of A
        Set<Integer> setA = new HashSet<>();
        for (int x : A) setA.add(x);
        for (int x : B) if (!setA.contains(x)) return false;
        return true;
    }
}
```

```
public static void main(String[] args) {  
    int[] A = {11,1,13,21,3,7};  
    int[] B = {11,3,7,1};  
    System.out.println(isSubset(A, B)); // true  
}  
}
```

Chapter 1 — Practice Problems

Attempt each problem before reading the hint.

1. Given {3, 1, 4, 1, 5, 9, 2, 6}, find the sum, average, and median.
2. Rotate {1,2,3,4,5,6,7} left by 3. Expected: {4,5,6,7,1,2,3}.
3. Find the equilibrium index of {1,3,5,2,2}. (Answer: 2, sum=4 on both sides).
4. Given {2,3,-2,4,-1}, find the maximum product subarray. (Answer: 48, subarray {-2,4,-1} no... let's check: $2*3=6$, $2*3*-2=-12$, $3*-2=-6$, $-2*4=-8$, $4*-1=-4$. So max=6. Actually full: $2*3=6$. Check $-2*4*-1=8$ also possible... depends on contiguity. 2,3,-2,4 -> $2*3*-2*4=-48$ no. Subarray 2,3 gives 6. Subarray 4 gives 4. Subarray -2,4,-1... wait $-2*4*-1=8$. Yes, 8 is max.)
5. Sort {4,2,2,8,3,3,1} by frequency descending. Expected: {2,2,3,3,4,8,1} or {3,3,2,2,1,4,8} (tie-break may vary).

Hints:

- P1: Sum=31, Avg $\approx$ 3.875, Median=sort first $\rightarrow$ {1,1,2,3,4,5,6,9} $\rightarrow$ (3+4)/2=3.5
- P3: leftSum at i=2 is 1+3=4; rightSum=2+2=4. Answer: index 2.
- P4: Track maxProd and minProd. At index 4 (x=-1): maxProd=-2*4*-1... trace through the algorithm.

Chapter 2: Numbers

Number-based problems test your mathematical reasoning and ability to implement mathematical formulas in code. They appear frequently in TCS NQT and are usually solvable in under 20 lines of code once you know the formula or trick.

2.1 Palindrome Number

A palindrome number reads the same forwards and backwards. E.g., 121, 1221, 12321.

Strategy: Reverse the digits and compare, OR use string reversal.

```
public class PalindromeNumber {
    public static boolean isPalindrome(int n) {
        if (n < 0) return false; // negative numbers are never palindromes
        int original = n, reversed = 0;
        while (n > 0) {
            reversed = reversed * 10 + n % 10;
            n /= 10;
        }
        return original == reversed;
    }

    // Find all palindromes in range [low, high]
    public static void palindromesInRange(int low, int high) {
        for (int i = low; i <= high; i++)
            if (isPalindrome(i)) System.out.print(i + " ");
    }

    public static void main(String[] args) {
        System.out.println(isPalindrome(121)); // true
        System.out.println(isPalindrome(123)); // false
        palindromesInRange(1, 50);
        // 1 2 3 4 5 6 7 8 9 11 22 33 44
    }
}
```

2.2 Prime Number Check

A prime number has exactly two factors: 1 and itself. The key insight for an efficient check is: you only need to test divisors up to $\sqrt{n}$. If n has a factor larger than $\sqrt{n}$, it must also have one smaller than $\sqrt{n}$.

```
public class Prime {
    public static boolean isPrime(int n) {
        if (n < 2) return false;
        if (n == 2) return true;
        if (n % 2 == 0) return false; // skip even numbers
        for (int i = 3; i * i <= n; i += 2) { // only odd divisors
```

```
        if (n % i == 0) return false;
    }
    return true;
}

// Sieve of Eratosthenes – find all primes up to 'limit' in O(n log log n)
public static boolean[] sieve(int limit) {
    boolean[] isPrime = new boolean[limit + 1];
    java.util.Arrays.fill(isPrime, true);
    isPrime[0] = isPrime[1] = false;
    for (int i = 2; i * i <= limit; i++) {
        if (isPrime[i]) {
            for (int j = i*i; j <= limit; j += i)
                isPrime[j] = false;
        }
    }
    return isPrime;
}

public static void main(String[] args) {
    System.out.println(isPrime(17));    // true
    System.out.println(isPrime(18));    // false

    // Print primes 1..50
    boolean[] p = sieve(50);
    for (int i = 2; i <= 50; i++) if (p[i]) System.out.print(i + " ");
    // 2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
}
}
```

2.3 Armstrong Number

An Armstrong number (also called Narcissistic number) is one where the sum of each digit raised to the power of the number of digits equals the number itself. E.g., $153 = 1^3 + 5^3 + 3^3 = 1 + 125 + 27 = 153$.

```
public class Armstrong {
    public static boolean isArmstrong(int n) {
        String s = Integer.toString(n);
        int power = s.length(); // number of digits
        int sum = 0, temp = n;
        while (temp > 0) {
            int digit = temp % 10;
            sum += (int) Math.pow(digit, power);
            temp /= 10;
        }
    }
}
```

```
        return sum == n;
    }

    public static void main(String[] args) {
        System.out.println(isArmstrong(153)); // true
        System.out.println(isArmstrong(370)); // true
        System.out.println(isArmstrong(371)); // true
        System.out.println(isArmstrong(9474)); // true (4 digits)
        System.out.println(isArmstrong(123)); // false
    }
}
```

2.4 Perfect Number

A perfect number equals the sum of all its proper divisors (divisors excluding itself). Example: $6 = 1 + 2 + 3$.

```
public class PerfectNumber {
    public static boolean isPerfect(int n) {
        if (n <= 1) return false;
        int sum = 1; // 1 is always a divisor
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                sum += i;
                if (i != n / i) sum += n / i; // avoid counting sqrt twice
            }
        }
        return sum == n;
    }

    public static void main(String[] args) {
        System.out.println(isPerfect(6)); // true (1+2+3=6)
        System.out.println(isPerfect(28)); // true (1+2+4+7+14=28)
        System.out.println(isPerfect(496)); // true
        System.out.println(isPerfect(12)); // false (1+2+3+4+6=16)
    }
}
```

2.5 GCD and LCM

GCD (Greatest Common Divisor) — Use Euclidean Algorithm: $\text{gcd}(a, b) = \text{gcd}(b, a \% b)$ until $b = 0$.

LCM (Least Common Multiple) — $\text{lcm}(a, b) = (a / \text{gcd}(a, b)) * b$ (divide first to avoid overflow).

```
public class GcdLcm {
    public static int gcd(int a, int b) {
        return b == 0 ? a : gcd(b, a % b);
    }
}
```

```

public static long lcm(int a, int b) {
    return (long) a / gcd(a, b) * b; // cast before multiply to avoid
    overflow
}

public static void main(String[] args) {
    System.out.println(gcd(12, 18)); // 6
    System.out.println(lcm(4, 6)); // 12
    System.out.println(gcd(100, 75)); // 25
    System.out.println(lcm(100, 75)); // 300
}
}

```

2.6 Fibonacci Sequence

Fibonacci: $F(0)=0$, $F(1)=1$, $F(n)=F(n-1)+F(n-2)$. Every TCS NQT paper has at least one Fibonacci variant.

```

public class Fibonacci {
    // Iterative – O(n) time, O(1) space
    public static void printFibonacci(int n) {
        long a = 0, b = 1;
        System.out.print(a + " " + b + " ");
        for (int i = 2; i < n; i++) {
            long c = a + b;
            System.out.print(c + " ");
            a = b; b = c;
        }
    }

    // Recursive – elegant but O(2^n) – DO NOT use for large n
    public static long fib(int n) {
        if (n <= 1) return n;
        return fib(n-1) + fib(n-2);
    }

    public static void main(String[] args) {
        printFibonacci(10);
        // 0 1 1 2 3 5 8 13 21 34
    }
}

```

FIBONACCI OVERFLOW

$F(46) = 1,836,311,903$ which is near int limit. $F(47)$ overflows int.

$F(93)$ overflows long. Use BigInteger for larger values.

For TCS NQT: use long and n will be small enough.

2.7 Factorial

```
import java.math.BigInteger;

public class Factorial {
    // Iterative
    public static long factorial(int n) {
        long result = 1;
        for (int i = 2; i <= n; i++) result *= i;
        return result;
    }

    // BigInteger for large n
    public static BigInteger factorialBig(int n) {
        BigInteger result = BigInteger.ONE;
        for (int i = 2; i <= n; i++) result =
            result.multiply(BigInteger.valueOf(i));
        return result;
    }

    public static void main(String[] args) {
        System.out.println(factorial(10));      // 3628800
        System.out.println(factorialBig(20));   // 2432902008176640000
        System.out.println(factorialBig(50));   // works fine with BigInteger
    }
}
```

2.8 Sum of AP and GP Series

AP (Arithmetic Progression): $a, a+d, a+2d, \dots$ Sum of n terms = $n/2 * (2a + (n-1)d)$

GP (Geometric Progression): $a, ar, ar^2, \dots$ Sum of n terms = $a*(r^n - 1)/(r-1)$ for $r \neq 1$, else $n*a$.

```
public class SeriesSum {
    public static double sumAP(double a, double d, int n) {
        return n / 2.0 * (2 * a + (n - 1) * d);
    }

    public static double sumGP(double a, double r, int n) {
        if (r == 1) return n * a;
        return a * (Math.pow(r, n) - 1) / (r - 1);
    }

    public static void main(String[] args) {
        System.out.println(sumAP(1, 2, 5));      // 1+3+5+7+9 = 25.0
        System.out.println(sumGP(1, 2, 5));      // 1+2+4+8+16 = 31.0
        System.out.println(sumGP(3, 1, 4));      // 3+3+3+3 = 12.0
    }
}
```

```
}  
}
```

2.9 Strong Number

A strong number is one where the sum of factorials of its digits equals the number itself. Example: $145 = 1! + 4! + 5! = 1 + 24 + 120 = 145$.

```
public class StrongNumber {  
    static int factorial(int n) {  
        int f = 1;  
        for (int i = 2; i <= n; i++) f *= i;  
        return f;  
    }  
  
    public static boolean isStrong(int n) {  
        int sum = 0, temp = n;  
        while (temp > 0) {  
            sum += factorial(temp % 10);  
            temp /= 10;  
        }  
        return sum == n;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(isStrong(1));    // true  (1! = 1)  
        System.out.println(isStrong(2));    // true  (2! = 2)  
        System.out.println(isStrong(145));  // true  
        System.out.println(isStrong(40585)); // true  
    }  
}
```

2.10 Automorphic Number

A number is automorphic if its square ends with the number itself. E.g., $5^2 = 25$ (ends in 5), $25^2 = 625$ (ends in 25).

```
public class Automorphic {  
    public static boolean isAutomorphic(int n) {  
        long sq = (long) n * n;  
        String s = Integer.toString(n);  
        String sqStr = Long.toString(sq);  
        return sqStr.endsWith(s);  
    }  
  
    public static void main(String[] args) {  
        System.out.println(isAutomorphic(5));    // true  (25)  
    }  
}
```

```

        System.out.println(isAutomorphic(25));    // true   (625)
        System.out.println(isAutomorphic(76));    // true   (5776)
        System.out.println(isAutomorphic(7));     // false  (49)
    }
}

```

2.11 Harshad Number

A Harshad (or Niven) number is divisible by the sum of its digits. Example: 18, digit sum = $1+8 = 9$, and $18 \% 9 = 0$.

```

public class Harshad {
    public static boolean isHarshad(int n) {
        int sum = 0, temp = n;
        while (temp > 0) { sum += temp % 10; temp /= 10; }
        return n % sum == 0;
    }

    public static void main(String[] args) {
        System.out.println(isHarshad(18));    // true
        System.out.println(isHarshad(19));    // false (digit sum=10, 19%10=9)
        System.out.println(isHarshad(21));    // true (digit sum=3, 21%3=0)
    }
}

```

2.12 Abundant Number

A number n is abundant if the sum of its proper divisors is greater than n . Example: 12 → divisors 1,2,3,4,6 → sum=16 > 12.

```

public class Abundant {
    public static boolean isAbundant(int n) {
        int sum = 1;
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                sum += i;
                if (i != n / i) sum += n / i;
            }
        }
        return sum > n;
    }

    public static void main(String[] args) {
        System.out.println(isAbundant(12));    // true
        System.out.println(isAbundant(18));    // true (1+2+3+6+9=21)
        System.out.println(isAbundant(15));    // false (1+3+5=9)
    }
}

```

```
}
```

2.13 More Number Problems

Reverse Digits of a Number

```
public class ReverseDigits {  
    public static int reverse(int n) {  
        boolean neg = n < 0;  
        if (neg) n = -n;  
        int rev = 0;  
        while (n > 0) { rev = rev*10 + n%10; n /= 10; }  
        return neg ? -rev : rev;  
    }  
    public static void main(String[] args) {  
        System.out.println(reverse(12345));    // 54321  
        System.out.println(reverse(-456));     // -654  
    }  
}
```

Max and Min Digit in a Number

```
public class MaxMinDigit {  
    public static void maxMin(int n) {  
        n = Math.abs(n);  
        int maxD = 0, minD = 9;  
        while (n > 0) {  
            int d = n % 10;  
            if (d > maxD) maxD = d;  
            if (d < minD) minD = d;  
            n /= 10;  
        }  
        System.out.println("Max digit: " + maxD + ", Min digit: " + minD);  
    }  
    public static void main(String[] args) { maxMin(382651); }  
    // Max digit: 8, Min digit: 1  
}
```

Sum of Digits

```
public class SumDigits {  
    public static int sumDigits(int n) {  
        n = Math.abs(n);  
        int sum = 0;  
        while (n > 0) { sum += n % 10; n /= 10; }  
        return sum;  
    }  
}
```

Power of a Number

```
public class Power {  
    // Fast exponentiation (Exponentiation by Squaring) – O(log exp)  
    public static long power(long base, int exp) {  
        long result = 1;  
        while (exp > 0) {  
            if ((exp & 1) == 1) result *= base; // if exp is odd  
            base *= base;  
            exp >>= 1; // exp /= 2  
        }  
        return result;  
    }  
    // Or simply: Math.pow(base, exp) returns double  
    public static void main(String[] args) {  
        System.out.println(power(2, 10)); // 1024  
        System.out.println(Math.pow(2, 10)); // 1024.0  
    }  
}
```

Leap Year Check

```
// A year is a leap year if:  
// - Divisible by 4 AND  
// - (NOT divisible by 100 OR divisible by 400)  
public static boolean isLeapYear(int year) {  
    return (year % 4 == 0) && (year % 100 != 0 || year % 400 == 0);  
}  
// isLeapYear(2000) -> true, isLeapYear(1900) -> false, isLeapYear(2024) -> true
```

Permutations: N people in R seats

$P(n, r) = n! / (n-r)!$ — number of ways n people can fill r ordered seats.

```
public static long permutations(int n, int r) {  
    if (r > n) return 0;  
    long result = 1;  
    for (int i = n; i > n - r; i--) result *= i; // n*(n-1)*...*(n-r+1)  
    return result;  
}  
// P(5,2) = 5*4 = 20
```

Adding Two Fractions

```
// a/b + c/d = (a*d + b*c) / (b*d), then simplify by GCD  
public static void addFractions(int a, int b, int c, int d) {  
    int num = a*d + b*c;  
    int den = b*d;  
    int g = gcd(Math.abs(num), Math.abs(den));
```

```

        System.out.println((num/g) + "/" + (den/g));
    }
    // addFractions(1,2,1,3) -> 3/6 + 2/6 = 5/6

```

Replace 0s with 1s

```

// Treat number as string, replace '0' chars with '1'
public static int replaceZeros(int n) {
    return Integer.parseInt(Integer.toString(n).replace('0', '1'));
}
// replaceZeros(102) -> 112

```

Can Number be Sum of Two Primes?

(Goldbach's conjecture — verified for all even numbers up to 4×10^{18}). For the exam: iterate from 2 to $n/2$, check if both i and $n-i$ are prime.

```

public static boolean sumOfTwoPrimes(int n) {
    for (int i = 2; i <= n/2; i++) {
        if (isPrime(i) && isPrime(n-i)) {
            System.out.println(n + " = " + i + " + " + (n-i));
            return true;
        }
    }
    return false;
}
// sumOfTwoPrimes(34) -> 34 = 3 + 31 (true)

```

Roots of a Quadratic Equation

```

// ax^2 + bx + c = 0
// discriminant D = b^2 - 4ac
// roots = (-b ± sqrt(D)) / 2a
public static void quadraticRoots(double a, double b, double c) {
    double D = b*b - 4*a*c;
    if (D > 0) {
        double r1 = (-b + Math.sqrt(D)) / (2*a);
        double r2 = (-b - Math.sqrt(D)) / (2*a);
        System.out.println("Two real roots: " + r1 + ", " + r2);
    } else if (D == 0) {
        System.out.println("One root: " + (-b / (2*a)));
    } else {
        double re = -b / (2*a);
        double im = Math.sqrt(-D) / (2*a);
        System.out.println("Complex roots: " + re + " ± " + im + "i");
    }
}

```

Chapter 2 — Practice Problems

6. Find all Armstrong numbers between 1 and 1000.
7. Find all perfect numbers below 10000. (6, 28, 496, 8128)
8. Print first 15 Fibonacci numbers.
9. Is 40585 a strong number? Verify step by step.
10. Find GCD and LCM of 56 and 98.

Hints: P4: $4!+0!+5!+8!+5! = 24+1+120+40320+120 = 40585$. Yes!

P5: GCD(56,98): $98=56*1+42$; $56=42*1+14$; $42=14*3+0 \rightarrow \text{GCD}=14$. LCM= $56*98/14=392$.

Chapter 3: Number Systems

Number system conversions are a staple of TCS NQT. Understand the theory, then memorise the pattern for each conversion. Java's built-in methods (`Integer.toBinaryString`, `Integer.parseInt` with radix) make many of these one-liners, but you must know the manual algorithm too.

3.1 Number System Basics

Decimal (Base 10): digits 0-9. Binary (Base 2): digits 0-1. Octal (Base 8): digits 0-7. Hexadecimal (Base 16): digits 0-9, A-F.

QUICK REFERENCE — Java Built-ins

Binary → Decimal: `Integer.parseInt("1010", 2) = 10`
 Decimal → Binary: `Integer.toBinaryString(10) = "1010"`
 Decimal → Octal: `Integer.toOctalString(8) = "10"`
 Decimal → Hex: `Integer.toHexString(255) = "ff"`
 Any base → Decimal: `Integer.parseInt(str, base)`
 Decimal → Any base: `Integer.toString(n, base)`

3.2 Binary ↔ Decimal

Binary to Decimal — Double-and-Add (manual)

Read binary string left to right. Start with `result=0`. For each bit: `result = result*2 + bit`.

```
public class BinaryDecimal {
    // Manual binary to decimal
    public static int binToDec(String bin) {
        int result = 0;
        for (char c : bin.toCharArray()) {
            result = result * 2 + (c - '0');
        }
        return result;
    }

    // Decimal to binary
    public static String decToBin(int n) {
        if (n == 0) return "0";
        StringBuilder sb = new StringBuilder();
        while (n > 0) { sb.append(n % 2); n /= 2; }
        return sb.reverse().toString();
    }

    public static void main(String[] args) {
        System.out.println(binToDec("1010"));           // 10
        System.out.println(decToBin(10));               // 1010
        // Built-in alternatives:
    }
}
```

```

        System.out.println(Integer.parseInt("1010",2)); // 10
        System.out.println(Integer.toBinaryString(10)); // 1010
    }
}

```

3.3 Binary ↔ Octal

Every 3 binary digits = 1 octal digit. Group binary digits from RIGHT in sets of 3, pad left with zeros if needed.

Example: 110101 → 110 101 → 6 5 → Octal: 65

```

public class BinaryOctal {
    public static String binToOct(String bin) {
        // Pad to multiple of 3
        while (bin.length() % 3 != 0) bin = "0" + bin;
        StringBuilder sb = new StringBuilder();
        for (int i = 0; i < bin.length(); i += 3) {
            String group = bin.substring(i, i+3);
            sb.append(Integer.parseInt(group, 2));
        }
        return sb.toString();
    }

    public static String octToBin(String oct) {
        StringBuilder sb = new StringBuilder();
        for (char c : oct.toCharArray()) {
            String b = Integer.toBinaryString(c - '0');
            // Pad to 3 digits
            while (b.length() < 3) b = "0" + b;
            sb.append(b);
        }
        // Remove leading zeros
        int i = 0;
        while (i < sb.length()-1 && sb.charAt(i) == '0') i++;
        return sb.substring(i);
    }

    public static void main(String[] args) {
        System.out.println(binToOct("110101")); // 65
        System.out.println(octToBin("65"));      // 110101
    }
}

```

3.4 Decimal ↔ Octal

```

public class DecimalOctal {

```

```

public static String decToOct(int n) {
    if (n == 0) return "0";
    StringBuilder sb = new StringBuilder();
    while (n > 0) { sb.append(n % 8); n /= 8; }
    return sb.reverse().toString();
}

public static int octToDec(String oct) {
    return Integer.parseInt(oct, 8);
}

public static void main(String[] args) {
    System.out.println(decToOct(64));           // 100
    System.out.println(octToDec("100"));        // 64
    // Built-ins:
    System.out.println(Integer.toOctalString(64)); // 100
}

```

3.5 Numbers to Words

Converting digits/numbers to words (e.g., 123 → "One Hundred Twenty Three") is a classic coding challenge.

```

public class NumberToWords {
    static String[] ones = {"", "One", "Two", "Three", "Four", "Five",
        "Six", "Seven", "Eight", "Nine", "Ten", "Eleven", "Twelve",
        "Thirteen", "Fourteen", "Fifteen", "Sixteen", "Seventeen",
        "Eighteen", "Nineteen"};
    static String[] tens = {"", "", "Twenty", "Thirty", "Forty", "Fifty",
        "Sixty", "Seventy", "Eighty", "Ninety"};

    public static String convert(int n) {
        if (n == 0) return "Zero";
        if (n < 0) return "Minus " + convert(-n);
        if (n < 20) return ones[n];
        if (n < 100) return tens[n/10] + (n%10 != 0 ? " " + ones[n%10] : "");
        if (n < 1000) return ones[n/100] + " Hundred" +
            (n%100 != 0 ? " " + convert(n%100) : "");
        if (n < 100000) return convert(n/1000) + " Thousand" +
            (n%1000 != 0 ? " " + convert(n%1000) : "");
        return "Large"; // extend as needed
    }

    public static void main(String[] args) {
        System.out.println(convert(19)); // Nineteen
    }
}

```

	System.out.println(convert(100)); // One Hundred
Four	System.out.println(convert(1234)); // One Thousand Two Hundred Thirty
	}
	}

Chapter 3 — Practice Problems

11. Convert 255 to binary, octal, hex. (Answers: 11111111, 377, ff)
12. Convert binary 10110111 to octal. (Group: 010 110 111 → 2 6 7 → 267)
13. Convert 1024 to words. (One Thousand Twenty Four)
14. What octal number equals decimal 500? (Integer.toOctalString(500) = 764)

Chapter 4: Sorting Algorithms

Sorting is fundamental to computer science. For TCS NQT, you must be able to both implement classic sorting algorithms from scratch and use Java's built-in sort intelligently. This chapter covers five core algorithms: Bubble, Selection, Insertion, Quick, and Merge Sort.

Algorithm	Best	Average	Worst	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$

4.1 Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. The largest element "bubbles up" to the end after each pass. It is the simplest sorting algorithm but $O(n^2)$ for large inputs.

Optimisation: If no swap occurs in a full pass, the array is already sorted — add a 'swapped' flag to achieve $O(n)$ best case.

```
import java.util.Arrays;

public class BubbleSort {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;
            for (int j = 0; j < n - 1 - i; j++) {
                if (arr[j] > arr[j+1]) {
                    int tmp = arr[j]; arr[j] = arr[j+1]; arr[j+1] = tmp;
                    swapped = true;
                }
            }
            if (!swapped) break; // already sorted
        }
    }

    public static void main(String[] args) {
        int[] arr = {64, 34, 25, 12, 22, 11, 90};
        bubbleSort(arr);
        System.out.println(Arrays.toString(arr));
        // [11, 12, 22, 25, 34, 64, 90]
    }
}
```

4.2 Selection Sort

Selection Sort divides the array into a sorted and unsorted region. It repeatedly finds the minimum element from the unsorted region and places it at the beginning of the unsorted region. Makes only $O(n)$ swaps — useful when write operations are costly.

```
public class SelectionSort {
    public static void selectionSort(int[] arr) {
        int n = arr.length;
        for (int i = 0; i < n - 1; i++) {
            int minIdx = i;
            for (int j = i + 1; j < n; j++) {
                if (arr[j] < arr[minIdx]) minIdx = j;
            }
            // Swap arr[i] and arr[minIdx]
            int tmp = arr[i]; arr[i] = arr[minIdx]; arr[minIdx] = tmp;
        }
    }

    // Trace on {5, 3, 1, 4, 2}:
    // Pass 1: min=1 at idx 2, swap with idx 0 -> {1, 3, 5, 4, 2}
    // Pass 2: min=2 at idx 4, swap with idx 1 -> {1, 2, 5, 4, 3}
    // Pass 3: min=3 at idx 4, swap with idx 2 -> {1, 2, 3, 4, 5}
}
```

4.3 Insertion Sort

Insertion Sort builds a sorted sub-array one element at a time. It takes the next unsorted element and inserts it into its correct position in the sorted portion. Excellent for nearly-sorted data and for small arrays (used as the base case in TimSort, Java's built-in sort).

```
public class InsertionSort {
    public static void insertionSort(int[] arr) {
        int n = arr.length;
        for (int i = 1; i < n; i++) {
            int key = arr[i];
            int j = i - 1;
            // Shift elements that are greater than key to the right
            while (j >= 0 && arr[j] > key) {
                arr[j+1] = arr[j];
                j--;
            }
            arr[j+1] = key;
        }
    }

    // Trace on {5, 1, 4, 2}:
    // i=1: key=1, shift 5 right -> {5,5,4,2}, insert 1 -> {1,5,4,2}
    // i=2: key=4, shift 5 right -> {1,5,5,2}, insert 4 -> {1,4,5,2}
    // i=3: key=2, shift 5,4 right, insert 2 -> {1,2,4,5}
}
```

}

4.4 Quick Sort

Quick Sort is a divide-and-conquer algorithm. It picks a 'pivot' element and partitions the array into elements smaller than the pivot (left) and larger (right), then recursively sorts both parts. Average $O(n \log n)$, worst $O(n^2)$ (when pivot is always min or max — avoidable with random pivot).

```
public class QuickSort {
    // Partition using Lomuto scheme (pivot = last element)
    private static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low - 1; // index of smaller element

        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                i++;
                int tmp = arr[i]; arr[i] = arr[j]; arr[j] = tmp;
            }
        }
        // Place pivot in correct position
        int tmp = arr[i+1]; arr[i+1] = arr[high]; arr[high] = tmp;
        return i + 1;
    }

    public static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int pi = partition(arr, low, high);
            quickSort(arr, low, pi - 1);
            quickSort(arr, pi + 1, high);
        }
    }

    public static void main(String[] args) {
        int[] arr = {10, 7, 8, 9, 1, 5};
        quickSort(arr, 0, arr.length - 1);
        // [1, 5, 7, 8, 9, 10]
        System.out.println(java.util.Arrays.toString(arr));
    }
}
```

QUICK SORT — WORST CASE AVOIDANCE

Worst case $O(n^2)$ occurs when array is already sorted and pivot=last element.

Fix: Use random pivot — swap `arr[random]` with `arr[high]` before partitioning.

Or use 'median of three' — pick median of first, middle, last as pivot.

4.5 Merge Sort

Merge Sort is a stable, divide-and-conquer sort. It splits the array into halves, recursively sorts each half, then merges the two sorted halves. Always $O(n \log n)$ — no worst case degradation.

```
public class MergeSort {
    private static void merge(int[] arr, int l, int m, int r) {
        // Create temp arrays
        int[] L = java.util.Arrays.copyOfRange(arr, l, m+1);
        int[] R = java.util.Arrays.copyOfRange(arr, m+1, r+1);

        int i = 0, j = 0, k = l;
        while (i < L.length && j < R.length) {
            if (L[i] <= R[j]) arr[k++] = L[i++];
            else arr[k++] = R[j++];
        }
        while (i < L.length) arr[k++] = L[i++];
        while (j < R.length) arr[k++] = R[j++];
    }

    public static void mergeSort(int[] arr, int l, int r) {
        if (l < r) {
            int m = l + (r - l) / 2;
            mergeSort(arr, l, m);
            mergeSort(arr, m+1, r);
            merge(arr, l, m, r);
        }
    }

    public static void main(String[] args) {
        int[] arr = {12, 11, 13, 5, 6, 7};
        mergeSort(arr, 0, arr.length - 1);
        System.out.println(java.util.Arrays.toString(arr));
        // [5, 6, 7, 11, 12, 13]
    }
}
```

Chapter 4 — Practice Problems

15. Trace Bubble Sort on {5,1,4,2,8}. Show the state after each pass.
16. How many swaps does Selection Sort make on {4,3,2,1}? (Answer: 2)
17. Implement Insertion Sort on a String array (sort alphabetically).
18. What is the pivot after the first partition of Quick Sort on {3,6,8,10,1,2,1} with pivot=last element (1)?

Hints: P4: pivot=1. All elements > 1, so i stays at -1. Swap arr[0] with arr[6] → {1,6,8,10,1,2,3}. pi=0.

Chapter 5: Strings

Strings are immutable in Java. Every string operation (concatenation, substring, etc.) creates a new String object. For repeated string modification, always use `StringBuilder`, which is mutable and $O(1)$ amortised for append.

5.0 String Basics and Key Methods

```
String s = "Hello, World!";

s.length()           // 13
s.charAt(0)           // 'H'
s.indexOf('o')        // 4
s.lastIndexOf('o')    // 8
s.substring(7)        // "World!"
s.substring(7, 12)    // "World"
s.toLowerCase()       // "hello, world!"
s.toUpperCase()       // "HELLO, WORLD!"
s.trim()              // removes leading/trailing spaces
s.replace('l', 'x')    // "Hexxo, Worxd!"
s.contains("World")    // true
s.startsWith("Hello") // true
s.endsWith("!")       // true
s.split(", ")         // ["Hello", "World!"]
s.toCharArray()      // char[]

// StringBuilder (mutable)
StringBuilder sb = new StringBuilder("hello");
sb.append(" world");   // "hello world"
sb.insert(5, ",");     // "hello, world"
sb.reverse();          // "dlrow ,olleh"
sb.deleteCharAt(0);    // "lrow ,olleh"
sb.toString();         // convert back to String
```

KEY RULE — String vs StringBuilder

String is immutable: `s = s + "x"` creates a NEW String object every time.

In a loop: `for(int i=0;i<1000;i++) s += 'a';` → $O(n^2)$ time and memory!

Always use `StringBuilder` when building a string in a loop:

```
StringBuilder sb = new StringBuilder();
for(int i=0;i<1000;i++) sb.append('a');
String result = sb.toString(); →  $O(n)$  time.
```

5.1 Check if a String is Palindrome

```
public class PalindromeString {
    // Method 1: Two-pointer
```

```
public static boolean isPalindrome(String s) {
    s = s.toLowerCase().replaceAll("[^a-z0-9]", ""); // ignore non-
    alphanumeric
    int l = 0, r = s.length() - 1;
    while (l < r) {
        if (s.charAt(l) != s.charAt(r)) return false;
        l++; r--;
    }
    return true;
}

// Method 2: StringBuilder reverse
public static boolean isPalindrome2(String s) {
    String rev = new StringBuilder(s).reverse().toString();
    return s.equals(rev);
}

public static void main(String[] args) {
    System.out.println(isPalindrome("racecar")); // true
    System.out.println(isPalindrome("A man a plan a canal Panama")); // true
    System.out.println(isPalindrome("hello")); // false
}
}
```

5.2 Count Vowels, Consonants, and Spaces

```
public class CountVCS {
    public static void count(String s) {
        int vowels = 0, consonants = 0, spaces = 0;
        s = s.toLowerCase();
        for (char c : s.toCharArray()) {
            if (c == ' ') spaces++;
            else if ("aeiou".indexOf(c) >= 0) vowels++;
            else if (c >= 'a' && c <= 'z') consonants++;
        }
        System.out.println("Vowels: " + vowels + ", Consonants: " +
            consonants + ", Spaces: " + spaces);
    }
    public static void main(String[] args) {
        count("Hello World");
        // Vowels: 3, Consonants: 7, Spaces: 1
    }
}
```

5.3 ASCII Value of a Character

```
// ASCII: 'A'=65, 'Z'=90, 'a'=97, 'z'=122, '0'=48, '9'=57, ' '=32
char c = 'A';
int ascii = (int) c;           // 65
char fromAscii = (char) 66;    // 'B'
// Lowercase to uppercase: c - 32
// Uppercase to lowercase: c + 32
// Digit char to int: c - '0' (e.g., '7' - '0' = 7)
```

5.4 Remove Vowels from String

```
public static String removeVowels(String s) {
    return s.replaceAll("[aeiouAEIOU]", "");
}
// removeVowels("Hello World") -> "Hll Wrld"

// Manual approach:
public static String removeVowelsManual(String s) {
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if ("aeiouAEIOU".indexOf(c) < 0) sb.append(c);
    }
    return sb.toString();
}
```

5.5 Remove Spaces from a String

```
public static String removeSpaces(String s) {
    return s.replace(" ", ""); // simple replace
    // or: s.replaceAll("\\s+", ""); // removes all whitespace (tabs, newlines too)
}
```

5.6 Remove Characters Except Alphabets

```
public static String onlyAlpha(String s) {
    return s.replaceAll("[^a-zA-Z]", "");
}
// onlyAlpha("He110 W0rld!") -> "HeWrld"
```

5.7 Reverse a String

```
public static String reverse(String s) {
    return new StringBuilder(s).reverse().toString();
}

// Manual two-pointer on char array:
```

```

public static String reverseManual(String s) {
    char[] c = s.toCharArray();
    int l = 0, r = c.length - 1;
    while (l < r) { char t = c[l]; c[l]=c[r]; c[r]=t; l++; r--; }
    return new String(c);
}

```

5.8 Remove Brackets from an Algebraic Expression

Remove all types of brackets: (), [], {} — but keep their contents. Simply filter out bracket characters.

```

public static String removeBrackets(String s) {
    return s.replaceAll("[()\\[\\]\\{\\}]", "");
}
// removeBrackets("(a+b)*[c-{d/e}]") -> "a+b*cd/e"

```

5.9 Sum of Numbers in a String

Extract all numeric sequences from the string and sum them.

```

public static int sumOfNumbers(String s) {
    int sum = 0;
    int i = 0;
    while (i < s.length()) {
        if (Character.isDigit(s.charAt(i))) {
            int num = 0;
            while (i < s.length() && Character.isDigit(s.charAt(i))) {
                num = num * 10 + (s.charAt(i) - '0');
                i++;
            }
            sum += num;
        } else {
            i++;
        }
    }
    return sum;
}
// sumOfNumbers("abc12def34") -> 46
// sumOfNumbers("12 and 34 plus 56") -> 102

```

5.10 Capitalize First and Last Character of Each Word

```

public static String capitalizeFirstLast(String s) {
    String[] words = s.split(" ");
    StringBuilder result = new StringBuilder();
    for (String word : words) {
        if (word.length() == 0) continue;
        if (word.length() == 1) {

```

```
        result.append(Character.toUpperCase(word.charAt(0)));
    } else {
        result.append(Character.toUpperCase(word.charAt(0)));
        result.append(word.substring(1, word.length()-1));
        result.append(Character.toUpperCase(word.charAt(word.length()-1)));
    }
    result.append(" ");
}
return result.toString().trim();
}
// capitalizeFirstLast("hello world") -> "Hello World"
```

5.11 Frequency of Characters in a String

```
import java.util.*;

public static Map<Character, Integer> charFrequency(String s) {
    Map<Character, Integer> freq = new LinkedHashMap<>();
    for (char c : s.toCharArray()) {
        freq.put(c, freq.getOrDefault(c, 0) + 1);
    }
    return freq;
}

// Alternatively, use int[256] for ASCII characters:
public static void charFreqArray(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    for (int i = 0; i < 256; i++)
        if (freq[i] > 0) System.out.println((char)i + " -> " + freq[i]);
}
```

5.12 Non-Repeating Characters of a String

```
public static void nonRepeating(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    System.out.print("Non-repeating: ");
    for (char c : s.toCharArray()) {
        if (freq[c] == 1) System.out.print(c + " ");
    }
}
// nonRepeating("programming") -> p o a i (first occurrence order)
```

5.13 Check if Two Strings are Anagrams

Two strings are anagrams if they contain the same characters with the same frequencies. Method: Sort both strings and compare, or use a frequency array.

```
import java.util.Arrays;

public class Anagram {
    // Method 1: Sort and compare – O(n log n)
    public static boolean isAnagram1(String a, String b) {
        if (a.length() != b.length()) return false;
        char[] ca = a.toLowerCase().toCharArray();
        char[] cb = b.toLowerCase().toCharArray();
        Arrays.sort(ca); Arrays.sort(cb);
        return Arrays.equals(ca, cb);
    }

    // Method 2: Frequency count – O(n)
    public static boolean isAnagram2(String a, String b) {
        if (a.length() != b.length()) return false;
        int[] count = new int[256];
        for (int i = 0; i < a.length(); i++) {
            count[a.charAt(i)]++;
            count[b.charAt(i)]--;
        }
        for (int c : count) if (c != 0) return false;
        return true;
    }

    public static void main(String[] args) {
        System.out.println(isAnagram1("listen", "silent")); // true
        System.out.println(isAnagram2("hello", "world"));  // false
    }
}
```

5.14 Wildcard Matching (* matches any sequence, ? matches one char)

Use dynamic programming or recursion. '?' matches any single character; '*' matches any sequence (including empty).

```
public static boolean wildcard(String text, String pattern) {
    int t = text.length(), p = pattern.length();
    boolean[][] dp = new boolean[t+1][p+1];
    dp[0][0] = true;

    // '*' at start matches empty text
    for (int j = 1; j <= p; j++)
        if (pattern.charAt(j-1) == '*') dp[0][j] = dp[0][j-1];
}
```

```

    for (int i = 1; i <= t; i++) {
        for (int j = 1; j <= p; j++) {
            char pc = pattern.charAt(j-1);
            if (pc == '*') {
                dp[i][j] = dp[i-1][j] || dp[i][j-1]; // match 1+ or 0
            } else if (pc == '?' || pc == text.charAt(i-1)) {
                dp[i][j] = dp[i-1][j-1];
            }
        }
    }
    return dp[t][p];
}

// wildcard("abcde", "a*e")    -> true
// wildcard("abcde", "a?cde") -> true
// wildcard("abcde", "a*f")    -> false

```

5.15 Maximum Occurring Character

```

public static char maxOccurring(String s) {
    int[] freq = new int[256];
    for (char c : s.toCharArray()) freq[c]++;
    int maxFreq = 0; char result = s.charAt(0);
    for (int i = 0; i < 256; i++) {
        if (freq[i] > maxFreq) { maxFreq = freq[i]; result = (char) i; }
    }
    return result;
}

// maxOccurring("programming") -> 'g' (appears 2 times, 'r' also 2 – first wins)

```

5.16 Remove All Duplicates from a String

```

public static String removeDups(String s) {
    boolean[] seen = new boolean[256];
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (!seen[c]) { sb.append(c); seen[c] = true; }
    }
    return sb.toString();
}

// removeDups("programming") -> "progamin"

```

5.17 Print All Duplicate Characters in a String

```

public static void printDuplicates(String s) {
    int[] freq = new int[256];

```

```
for (char c : s.toCharArray()) freq[c]++;
System.out.print("Duplicates: ");
for (int i = 0; i < 256; i++) {
    if (freq[i] > 1) System.out.print((char)i + "(" + freq[i] + ") ");
}
// printDuplicates("programming") -> g(2) m(2) r(2)
```

5.18 Remove Characters from First String Present in Second

```
public static String removeChars(String s, String remove) {
    Set<Character> toRemove = new HashSet<>();
    for (char c : remove.toCharArray()) toRemove.add(c);
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray())
        if (!toRemove.contains(c)) sb.append(c);
    return sb.toString();
}
// removeChars("Hello World", "lo") -> "He Wrld"
```

5.19 Change Every Letter to Next Lexicographic Alphabet

```
public static String nextLex(String s) {
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (c == 'z') sb.append('a');
        else if (c == 'Z') sb.append('A');
        else if (Character.isLetter(c)) sb.append((char)(c+1));
        else sb.append(c);
    }
    return sb.toString();
}
// nextLex("hello") -> "ifmmp"
// nextLex("xyz") -> "yza"
```

5.20 Largest Word in a String

```
public static String largestWord(String s) {
    String[] words = s.trim().split("\\s+");
    String largest = "";
    for (String w : words) {
        if (w.length() > largest.length()) largest = w;
    }
    return largest;
}
// largestWord("I love programming") -> "programming"
```

5.21 Sort Characters in a String

```
import java.util.Arrays;

public static String sortChars(String s) {
    char[] c = s.toCharArray();
    Arrays.sort(c);
    return new String(c);
}
// sortChars("programming") -> "aggimnopr"
```

5.22 Count Number of Words in a String

```
public static int countWords(String s) {
    s = s.trim();
    if (s.isEmpty()) return 0;
    return s.split("\\s+").length;
}
// countWords(" Hello World ") -> 2
```

5.23 Word with Highest Number of Repeated Letters

```
public static String wordWithMostRepeats(String s) {
    String[] words = s.trim().split("\\s+");
    String result = "";
    int maxRepeats = 0;

    for (String w : words) {
        int[] freq = new int[256];
        for (char c : w.toCharArray()) freq[c]++;
        int repeats = 0;
        for (int f : freq) if (f > 1) repeats += f;
        if (repeats > maxRepeats) { maxRepeats = repeats; result = w; }
    }
    return result;
}
// wordWithMostRepeats("I love programming") -> "programming" (r,m,g,n each 2)
```

5.24 Change Case of Each Character

```
public static String changeCase(String s) {
    StringBuilder sb = new StringBuilder();
    for (char c : s.toCharArray()) {
        if (Character.isUpperCase(c)) sb.append(Character.toLowerCase(c));
        else if (Character.isLowerCase(c)) sb.append(Character.toUpperCase(c));
        else sb.append(c);
    }
}
```

```
}  
    return sb.toString();  
}  
// changeCase("Hello World") -> "hELLO wORLD"
```

5.25 Concatenate One String to Another

```
// Built-in: s1 + s2 or s1.concat(s2)  
// For many concatenations, use StringBuilder:  
StringBuilder sb = new StringBuilder(s1);  
sb.append(s2);  
String result = sb.toString();
```

5.26 Find Substring Within a String

```
public static int findSubstring(String text, String pattern) {  
    return text.indexOf(pattern); // returns -1 if not found  
}  
// findSubstring("Hello World", "World") -> 6  
  
// Manual KMP or Brute force for educational purposes:  
public static int bruteForce(String text, String pattern) {  
    int n = text.length(), m = pattern.length();  
    for (int i = 0; i <= n - m; i++) {  
        int j = 0;  
        while (j < m && text.charAt(i+j) == pattern.charAt(j)) j++;  
        if (j == m) return i;  
    }  
    return -1;  
}
```

5.27 Reverse Words in a String

```
public static String reverseWords(String s) {  
    String[] words = s.trim().split("\\s+");  
    StringBuilder sb = new StringBuilder();  
    for (int i = words.length - 1; i >= 0; i--) {  
        sb.append(words[i]);  
        if (i != 0) sb.append(" ");  
    }  
    return sb.toString();  
}  
// reverseWords("Hello World Java") -> "Java World Hello"
```

5.28 Count Common Subsequences in Two Strings

A subsequence is obtained by deleting some characters without changing the order. Use Dynamic Programming.

```
public static int commonSubseq(String s1, String s2) {
    int m = s1.length(), n = s2.length();
    int[][] dp = new int[m+1][n+1];
    for (int i = 1; i <= m; i++) {
        for (int j = 1; j <= n; j++) {
            if (s1.charAt(i-1) == s2.charAt(j-1))
                dp[i][j] = 1 + dp[i-1][j-1];
            else
                dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]);
        }
    }
    return dp[m][n]; // Length of LCS (Longest Common Subsequence)
}

// commonSubseq("ABCBADAB", "BDCAB") -> 4 (BCAB or BDAB)
```

Chapter 5 — Practice Problems

19. Reverse the words in "The quick brown fox". Expected: "fox brown quick The".
20. Check if "Astronomer" and "Moon staler" are anagrams (ignore spaces). (Yes!)
21. Find the frequency of each character in "mississippi".
22. Remove duplicates from "banana". Expected: "ban".
23. Find the largest word in "Java is a programming language". Expected: "programming".
24. Apply nextLex() to "xyz". Expected: "yza".

Hint P3: $m \rightarrow 4$, $i \rightarrow 4$, $s \rightarrow 4$, $p \rightarrow 2$.

Hint P2: Remove spaces from both, sort chars, compare.

Chapter 6: Algorithm Patterns and Exam Strategies

This chapter consolidates the common algorithmic patterns used throughout the manual. Recognising these patterns is often what separates a candidate who solves the problem in 10 minutes from one who struggles for 40.

6.1 The Two-Pointer Technique

Use two pointers (left and right, or slow and fast) that move based on a condition. Reduces $O(n^2)$ brute force to $O(n)$.

When to use: Sorted array pair problems, palindrome check, reversing, removing duplicates.

```
// Template: Two pointers converging
int left = 0, right = arr.length - 1;
while (left < right) {
    // process arr[left] and arr[right]
    if (condition_to_move_left) left++;
    else if (condition_to_move_right) right--;
    else { left++; right--; }
}
```

6.2 Sliding Window

Maintain a 'window' of elements using two pointers. Expand the window by moving the right pointer; shrink by moving the left. Useful for subarray/substring problems asking for maximum, minimum, or exact conditions.

```
// Fixed-size window of size k
int windowSum = 0;
for (int i = 0; i < k; i++) windowSum += arr[i];
int maxSum = windowSum;
for (int i = k; i < arr.length; i++) {
    windowSum += arr[i] - arr[i-k]; // add new, remove old
    maxSum = Math.max(maxSum, windowSum);
}
```

6.3 Hashing for $O(1)$ Lookup

HashMap and HashSet are your best friends in TCS NQT. Whenever you need to check membership, count occurrences, or find if you've seen something before — use a Hash structure.

```
// Common HashMap patterns:

// 1. Frequency count
Map<T, Integer> freq = new HashMap<>();
for (T item : collection) freq.put(item, freq.getOrDefault(item, 0) + 1);

// 2. Seen-before check
```

```
Set<T> seen = new HashSet<>();
for (T item : collection) {
    if (seen.contains(item)) { /* duplicate found */ }
    seen.add(item);
}

// 3. Complement lookup (e.g., Two Sum)
Map<Integer, Integer> indexMap = new HashMap<>();
for (int i = 0; i < arr.length; i++) {
    if (indexMap.containsKey(target - arr[i])) { /* found pair */ }
    indexMap.put(arr[i], i);
}
```

6.4 Prefix Sums

Build a prefix sum array where $\text{prefix}[i] = \text{arr}[0] + \text{arr}[1] + \dots + \text{arr}[i-1]$. Then any range sum $\text{arr}[l..r] = \text{prefix}[r+1] - \text{prefix}[l]$ in $O(1)$.

```
int[] prefix = new int[arr.length + 1];
for (int i = 0; i < arr.length; i++)
    prefix[i+1] = prefix[i] + arr[i];

// Sum from index l to r (inclusive):
int rangeSum = prefix[r+1] - prefix[l];

// Equilibrium: left sum = prefix[i], right sum = prefix[n] - prefix[i+1]
```

6.5 Java Input/Output Quick Reference

```
import java.util.Scanner;

Scanner sc = new Scanner(System.in);

// Reading input:
int n    = sc.nextInt();
long l   = sc.nextLong();
double d = sc.nextDouble();
String s = sc.next();           // reads one word (no spaces)
String line = sc.nextLine();    // reads entire line

// Reading an array:
int n = sc.nextInt();
int[] arr = new int[n];
for (int i = 0; i < n; i++) arr[i] = sc.nextInt();

// Faster output – use println for TCS NQT (usually fine)
```

```
System.out.println("Result: " + result);
System.out.printf("%.2f%n", avg); // formatted output
```

6.6 Common Mistakes and How to Avoid Them

- Off-by-one errors: Always verify loop bounds. Is it $< n$ or $\leq n$? Dry-run on a small example.
- Integer overflow: For sums/products of large numbers, use long. For very large factorials, use BigInteger.
- Array index out of bounds: Check `arr[i-1]` and `arr[i+1]` accesses are within `[0, n-1]`.
- Null pointer on HashMap: Use `getOrDefault()` instead of `get()` to avoid NPE.
- Not handling empty input: Check `arr.length == 0` or `str.isEmpty()` at the start of your method.
- String vs Character comparison: Use `s.equals(t)`, not `s == t`. Use `c1 == c2` for chars (primitives).
- `Scanner.nextLine()` after `nextInt()`: `nextInt()` leaves a newline in the buffer. Call `sc.nextLine()` once to consume it before reading the next full line.

6.7 Time Complexity Cheat Sheet

Operation	Complexity	Notes
Linear scan	$O(n)$	Find min, max, sum, frequency
Binary search	$O(\log n)$	Sorted array required
Sorting	$O(n \log n)$	<code>Arrays.sort()</code> , Merge, Quick
HashMap lookup	$O(1)$ avg	Collision: $O(n)$ worst
Nested loops	$O(n^2)$	Brute-force pair search
Sieve of Eratosthenes	$O(n \log \log n)$	All primes up to n
Fibonacci (iterative)	$O(n)$	$O(2^n)$ for naive recursive
GCD (Euclidean)	$O(\log \min(a,b))$	Very fast
Factorial (iterative)	$O(n)$	Overflow: use long/BigInteger

6.8 Last-Minute Exam Checklist

25. Read the problem twice. Note: is output a single value, an array, or multiple lines?
26. Identify the pattern: linear scan / two-pointer / hashing / sort / math.
27. Write a brute-force solution first, then optimise.
28. Handle edge cases: empty input, $n=1$, all same elements, negative numbers.
29. Check overflow: if n and values are large, use long.
30. Dry-run your code mentally on the given example before submitting.
31. Even partial solutions get marks — submit something that works for basic test cases.

FINAL NOTE

Practice is the only shortcut. After reading this manual, solve each problem in the TCS NQT coding sheet on your own, referring back to the relevant section only when stuck. Aim to solve each problem in under 20 minutes. With consistent practice across all five chapters, you will be fully prepared for the exam.

Good luck!